PowerLab (SML Version)

Danny, Dani, Daniel, and Danielle have scheduled a Mooz meeting to prepare the upcoming final exam, when all of a sudden... all of the power in Mooville went out! Panicking, the four bovines have decided to take matters into their own hands and fix the power lines all by themselves. Given the high-steaks of the situation, your task is to help our beloved cows restore connectivity between their respective barns as soon as possible!

1 Files

After downloading the assignment tarball from Ed, extract the files by running:

```
tar -xvf powerlab-handout.tgz
```

from a terminal window.

Some of the files worth looking at are listed below. The files denoted by * will be handed in by the submission script.

1. `Makefile`
2. `*MkPower.sml`
3. `Sandbox.sml`
4. `Tests.sml`

Additionally, you should create a file called `written.pdf` which contains the answers to the written parts of the assignment.

2 Submission

To submit the code portion of the assignment, run `make package` and upload the resulting `handin.zip` to the Gradescope code folder. You should ensure your code compiles by checking for a score of 0 under the `Compilation` section on Gradescope. Note that Gradescope can take a few minutes to compile code submissions.

Submit the written portion by uploading your `written.pdf` to the Gradescope written folder.

3 Problem Setup

The power grid of Mooville is quite straightforward and can be modeled as a **weighted, connected, undirected, simple** (no self-loops; at most one edge between any two vertices) graph $G = (V, E)$, where each vertex represents a barn and the edges represent power lines between the barns. Additionally, because the power lines differ in length and material, each one takes a different amount of time to fix.

Formally, each power line can be modeled as a triple (u, v, w) where u and v are the barns it connects and w is a nonnegative number representing how long it takes to fix it (in minutes). The barns are labeled from 0 to $|V|-1$, where $|V|$ is the maximum barn label in the edge sequence, plus one.

3.1 Implementation

Task (1)**.** (0 points) In `MkPower.sml`, define the type `ugraph` representing an undirected graph and implement the following function.

```
val makeGraph : edges -> ugraph
```

Given a sequence E representing the edges of a graph G as described above, `makeGraph E` returns that same graph under your `ugraph` representation. This will be used in task 2.

Task (2)**.** (85 points) In `MkPower.sml`, implement the following function.

```
val findMinCost : ugraph -> (vertex * vertex * vertex * vertex) -> int
```

Given an undirected graph G and the locations of Danny, Dani, Daniel, and Danielle (say (a, b, c, d)), `findMinCost G (a, b, c, d)` returns the minimum possible total time (in minutes) to connect barns a, b, c, and d. More formally, consider a set of edges $E' \subseteq E$ such that barns (a, b, c, d) are all pairwise connected via only edges in E'. We want to find the minimum total cost of any such E'.

For full credit, both your `makeGraph` and `findMinCost` functions should have $O(n^3)$ work and $O(n)$ span.

Note. (a, b, c, d) are not necessarily pairwise unique!

Hint. You should make use of the argument structure ascribing to `ST_SEQUENCE`. Note that `ST_SEQUENCE` has a `Seq` substructure ascribing to `SEQUENCE`. You may assume the costs of `MkSTSequence` for the argument structure, and `ArraySequence` for the `Seq` substructure.

Note. For up to 10 points of extra credit, stage `findMinCost` to avoid performing unnecessary recomputation. In your code, comment the work/span of `findMinCost G`. You will receive credit only if your work/span bounds are as good as our reference solution.

3.2 Testing

It is very important that you thoroughly test your code before you submit. After the deadline, we will grade your code with a private set of test cases.

We provide two ways to test your code with SML/NJ.

1. In `Sandbox.sml`, write whatever testing code you'd like. You can then access the sandbox at the REPL:

   ```
   - CM.make "sandbox.cm";
   ...
   - open Sandbox;
   ```

2. In `Tests.sml`, add test cases according to the instructions given. Then run the autograder:

   ```
   - CM.make "autograder.cm";
   ...
   - Autograder.run ();
   ```

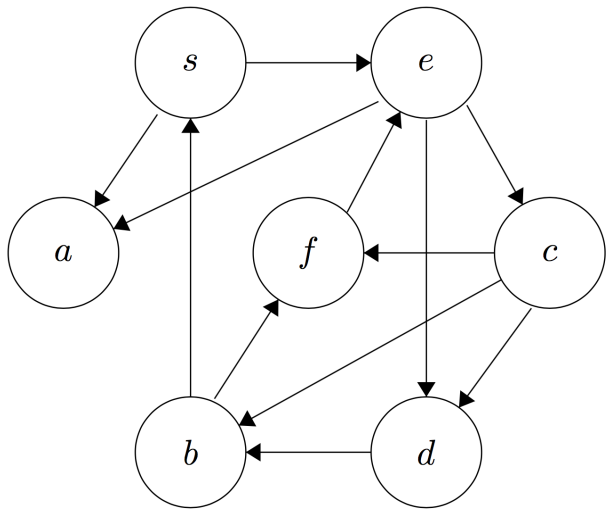
4 Written Questions

Having solved the problem, you've successfully brought Danny, Dani, Daniel, and Danielle back online in the fastest way possible, and they are truly grateful for your help! Maybe they'll make the exam ques-

tions a little bit easier as a form of gratitude... For now, let's make sure you're prepared by solving the following written problems!

4.1 3edgy5me

DFS numbering is a handy way to identify different types of edges in a graph. For the following two problems, refer to the graph below.



Task 1: Written (10 points)

Draw the DFS tree for the graph, starting at node s. If there are multiple possible nodes to expand at a step, expand the nodes in alphabetical order. Also give the DFS numbering (visit time, finish time) for each node, for example $s : (0, 13)$. Include only the tree edges in your DFS tree.

Remark. You can use `http://madebyevan.com/fsm/` to automatically generate LaTeX code for graphs that you've drawn.

Task 2: Written (5 points)

List all the forward edges, back edges, and cross edges in the graph.

For any directed graph G, we define its transpose, G^T, to be the graph that contains the same set of vertices with all of its edges reversed. That is, (u, v) exists in $G \longleftrightarrow (v, u)$ exists in G^T. We also define G^A, the autotranspose graph of G, as follows: $V(G^A) = V(G)$, $E(G^A) = E(G) \cup E(G^T)$.

Task 3: Written (5 points)

Give an example of a graph G with at most 4 vertices, such that there exists a *cross edge* $e = (u, v)$ in the DFS tree of G, but e (not its reverse) is a *back edge* in the DFS tree of G^A. Label your vertices alphabetically from "a". The DFS tree should start from the node labelled "a". If there are multiple possible nodes to expand at a step, do so in alphabetical order. Label the edge e clearly.

Task 4: Written (5 points)

Give an example of a graph H with at most 4 vertices, such that there exists a *tree edge* $e = (u, v)$ in the DFS tree of H, but e is a *back edge* in the DFS tree of H^A. Label your vertices alphabetically from "a". The DFS tree should start from the node labelled "a". If there are multiple possible nodes to expand at a step, do so in alphabetical order. Label the edge e clearly.

4.2 Connected Components

In an undirected graph, a connected component is a set of vertices such that for every pair vertices in the set, there exists a path between the two.

Task 5: Written (5 points)

Describe an algorithm that uses DFS to count the number of connected components in a undirected graph. You may give your answer in English or pseudocode. Your algorithm should run with $O(n + m)$ work and span. Please also justify the correctness of your algorithm.

4.3 Bipartite Graphs

Define a directed bipartite graph to be a graph where the vertices can be split into two sets A and B such that every edge has one endpoint in A and the other endpoint in B. Specifically, for every directed edge (x, y), either $(x \in A$ and $y \in B)$ or $(x \in B$ and $y \in A)$.

Task 6: Written (10 points)

Give an algorithm based on DFS to determine the two vertex sets A and B of a bipartite graph. Your algorithm should cost $O(n)$ work and span (i.e., it is a sequential algorithm).

Note. You may use C++ or SML or pseudocode for your answer.

4.4 Algorithm Design

Task 7: Written (20 points)

You're walking around an island in the Caribbean and stumble upon a treasure hideout, owned by a band of pirates. You see n transparent treasure chests, which contain keys made out of gold. Each key is labeled with a corresponding treasure chest that it opens. Each key will unlock at most one chest. Each chest could be unlocked by more than one key (or no keys!). Each chest can contain more than 1 key. There are m keys in total.

You're a very greedy person, and want to steal all the gold keys that are in the chests. There are only two ways to open each chest: unlock it with a proper key, or smash it with a hammer. You'll have to smash at least one chest, but it makes a pretty loud sound and you don't want to draw the attention of the pirates.

Describe an efficient algorithm with $O(m+n)$ work and span to compute the minimum number of chests you need to break with the hammer in order to open all the n chests. Please also justify the correctness of your algorithm.

Note. You only need to provide a high-level description of your algorithm in a few sentences: we don't need code or pseudocode.

Note. This problem is courtesy of Jeff Erickson, whose algorithms textbook you can find here. Feel free to check out this book: there are a lot of good algorithm design problems. You won't find the solutions anywhere but you are encouraged to come to office hours to discuss these problems!